

Prolog Program for Backward Chaining

1. Aim

To write and execute a Prolog program that implements backward chaining inference, proving a target goal by recursively reducing it to sub-goals and known facts, and to incorporate the required queries.

2. Objective

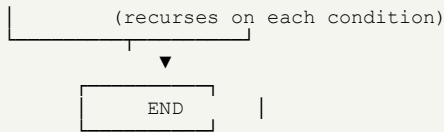
- To understand the backward chaining strategy of inference (goal-driven reasoning).
- To represent known facts and if-then rules with condition lists in Prolog.
- To implement a prove/1 predicate that attempts to satisfy a goal from facts or by proving all conditions of a matching rule.
- To retrieve whether a target goal can be proven true using a query.

3. Algorithm

1. Start the program.
2. Create a Prolog database of known facts using the predicate fact/1.
3. Create a set of rules using the predicate rule/2, each linking a goal to a list of conditions.
4. Define prove/1: a goal is proven if it is directly a known fact.
5. Otherwise, a goal is proven if some rule concludes it and all of that rule's conditions can also be proven.
6. Enter the target goal as a query to prove/1.
7. Prolog works backward from the goal, recursively proving each sub-condition.
8. If every condition succeeds, display that the goal is proven; otherwise Prolog returns false.
9. Stop the program.

4. Flowchart





5. Code with Output

Code:

```
% -----  
% Known facts  
% -----  
fact(has_feathers(tweety)).  
fact(lays_eggs(tweety)).  
  
% -----  
% Rules: goal :- [conditions]  
% -----  
rule(bird(X),      [has_feathers(X)]).  
rule(can_fly(X),   [bird(X)]).  
rule(oviparous(X), [lays_eggs(X)]).  
  
% -----  
% Backward chaining engine  
% -----  
prove(Goal) :-  
    fact(Goal).  
prove(Goal) :-  
    rule(Goal, Conditions),  
    prove_all(Conditions).  
  
prove_all([]).  
prove_all([H|T]) :-  
    prove(H),  
    prove_all(T).
```

Output:

```
?- prove(can_fly(tweety)).  
true.  
  
?- prove(oviparous(tweety)).  
true.  
  
?- prove(can_fly(rex)).  
false.
```

6. Result

The Prolog program was successfully implemented to perform backward chaining, proving that 'tweety can fly' by recursively reducing the goal to its supporting sub-goals and matching them against known facts.

7. Conclusion

Thus, backward chaining was successfully implemented using Prolog. The experiment demonstrates goal-driven inference: Prolog's own resolution strategy is used explicitly through `prove/1`, reducing a target goal to sub-goals until each is grounded in a known fact.